

1 The motor subsystem

The motor subsystem is a translation layer: it turns catalog motor data — a burn’s worth of measured thrust samples plus manufacturer metadata — into the two time functions the simulation core consumes during the powered ascent of ??: thrust $F(t)$ in newtons and motor mass $m(t)$ in kilograms. Thrust enters the force model as a body- z force through the center of mass `model/RocketModel.cpp:71`; mass enters through the part tree, where a `part::Motor` leaf reports the burning motor’s mass into the composite walk of ?? `model/parts/Motor.h:40`. Everything else here — piecewise-linear curves, a derived mass table, a keyed store, three ingest paths — exists to produce those two functions reliably from three interchange formats. [Figure 1](#) maps the classes.

What a hobby motor ships as data

A commercial model-rocket motor is a sealed solid-propellant cartridge, certified with a measured thrust-versus-time curve, a total impulse $I_{tot} = \int F dt$ in newton-seconds (N s), a propellant mass, and a loaded mass. The catalog grammar is compact: a code like “G80T” reads as impulse class G, average thrust roughly 80 N, propellant type letter T. Impulse classes are letters, each doubling the ceiling of the one before — A is at most 2.5 N s, B at most 5, up through M at 10240 N s — with fractional classes 1/4A and 1/2A below A. Motors also list *delay* options: seconds between burnout and the recovery-deploying ejection charge, with “plugged” (no charge) as a special case.

1.1 The runtime model: `MotorModel`

`model::MotorModel` `model/MotorModel.h:26` is a plain value type — copyable, movable, owned by value everywhere: in the database’s map, in the `part::Motor` tree leaf, in the loaders’ output vectors. It composes three pieces of state: a public `MetaData` record of catalog fields `model/MotorModel.h:285` ([Table 1](#); public with a standing `TOD0` to privatize `model/MotorModel.h:282`), a `ThrustCurve` owned by value `model/MotorModel.h:343`, and a derived `massCurve` — a 128-point table of remaining propellant versus burn time `model/MotorModel.h:345`, recomputed whenever metadata is set ([Section 1.3](#)).

The unit discipline deserves one explicit sentence, because it is the one place QtRocket’s SI-throughout convention (??) admits a deliberate exception. Weights are kilograms *at the `MotorModel` boundary* — each ingest path normalizes from its source’s native unit ([Table 2](#)) — while `diameter` and `length` stay in millimeters because every source catalog quotes them that way; the one consumer that needs meters, the `part::Motor` inertia tensor, converts at the point of use `model/parts/Motor.cpp:27`. A regression pins the kilogram contract: a 1/4A micro-rocket flown from an online-sourced database must lift off with an ignition mass under 0.05 kg, which the gram-as-kilogram failure mode (a 6.1 kg “motor”) cannot do `tests/DesignMatrixTests.cpp:269`.

1.2 `ThrustCurve`: piecewise-linear thrust

`ThrustCurve` `model/ThrustCurve.h:12` — the one motor type in the global namespace rather than `model::` — stores the samples as a `std::vector<std::pair<double, double>>` of (time s, thrust N) pairs `model/ThrustCurve.h:38` plus `maxTime`, the largest sample time `model/ThrustCurve.cpp:24`. A default-constructed curve is a single (0, 0) point, and an empty sample vector degenerates to the same, “so `getThrust()` always has an interval to walk” `model/ThrustCurve.cpp:19`.

`getThrust(t)` `model/ThrustCurve.cpp:47` first shifts into motor-relative time (`t -= ignitionTime`; `model/ThrustCurve.cpp:50` — always a zero shift in practice, [Section 1.4](#)), then clamps: outside `[0, maxTime]` thrust is identically zero `model/ThrustCurve.cpp:51`. No thrust before ignition, none after

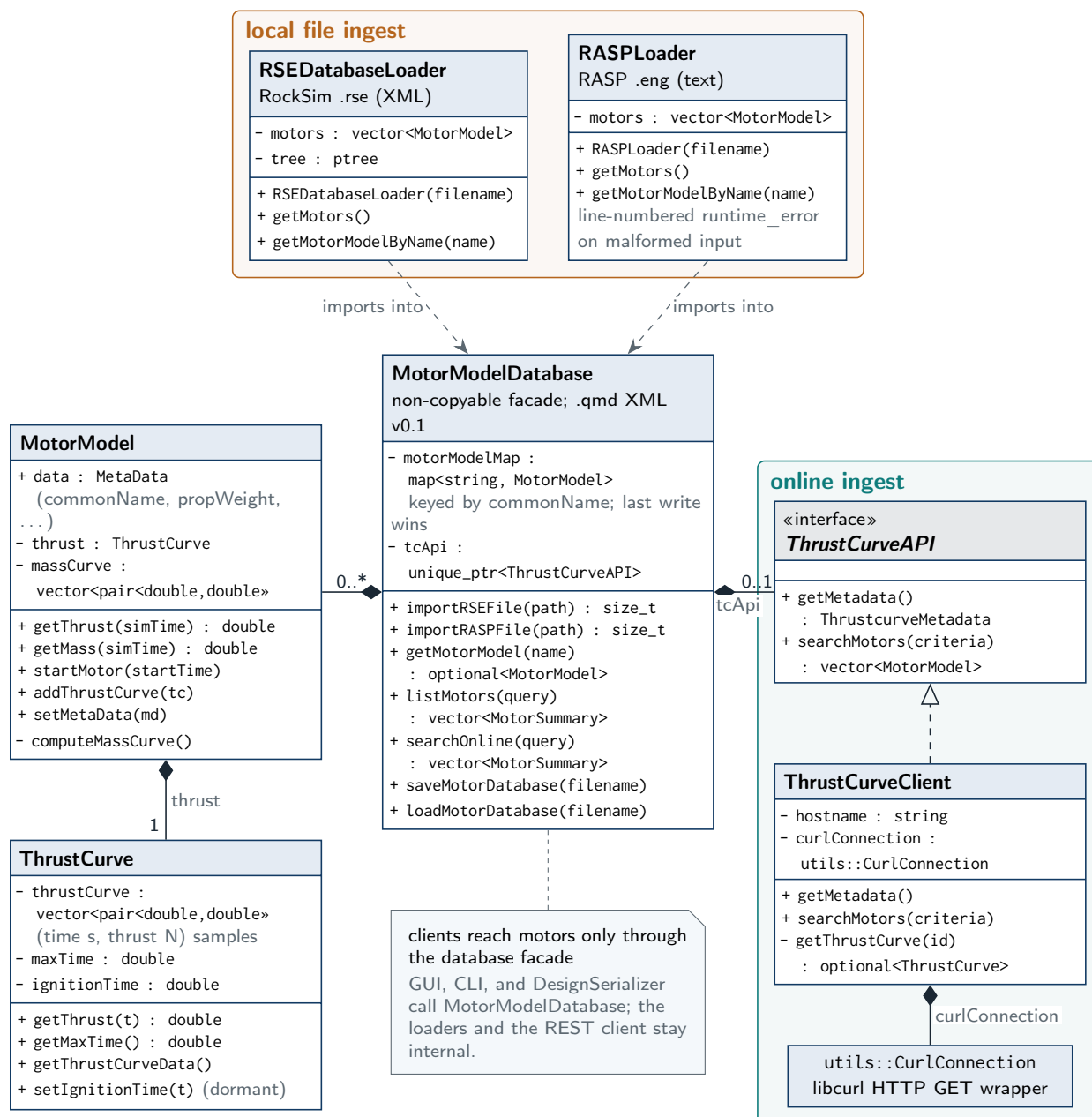


Figure 1. The motor subsystem. `MotorModelDatabase` is the single client-facing surface: a `std::map` of `MotorModel` values keyed by common name `model/MotorModelDatabase.h:127`. Two local file loaders — RockSim .rse and RASP .eng — are constructed stack-locally inside the import methods `model/MotorModelDatabase.cpp:66`, and the thrustcurve.org REST client sits behind the abstract `ThrustCurveAPI` seam `model/ThrustCurveClient.h:68`, injectable only through a private, friend-only constructor. Each `MotorModel` owns one `ThrustCurve` by value `model/MotorModel.h:343` and derives a 128-point propellant mass curve from it.

Field	Units	Meaning	
<code>commonName</code>	—	the database key; e.g. “A8”, “G80T”	model/MotorModel.h:301
<code>impulseClass</code>	—	motor letter: “1/4A”, “1/2A”, “A”...“M”	model/MotorModel.h:306
<code>diameter, length</code>	mm	casing geometry — deliberately <i>not</i> SI; hobby-catalog convention	model/MotorModel.h:305
<code>propWeight</code>	kg	propellant mass; “loaders convert; consumed as kg”	model/MotorModel.h:314
<code>totalWeight</code>	kg	loaded motor mass including casing	model/MotorModel.h:317
<code>totalImpulse</code>	N s	certified total impulse	model/MotorModel.h:316
<code>avgThrust, maxThrust</code>	N	catalog thrust summary values	model/MotorModel.h:298
<code>burnTime</code>	s	burn duration; gates the mass model	model/MotorModel.h:299
<code>delays</code>	s	ejection-delay options; 1000 is the plugged sentinel	model/MotorModel.h:303
<code>manufacturer, type, availability, certOrg</code>	—	enum wrappers, each with a <code>str()/toEnum()</code> string round trip	model/MotorModel.h:309
<code>motorIdTC</code>	—	24-character thrustcurve.org hex identifier	model/MotorModel.h:312

Table 1. Selected `MotorModel::MetaData` fields [model/MotorModel.h:285](#). The units column is the SI story in miniature: weights are kilograms by documented contract — every ingest path normalizes to kg at the boundary — while `diameter` and `length` stay in millimeters, converted to meters only where physics needs them [model/parts/Motor.cpp:27](#).

the last sample — after burnout the rocket coasts until the termination condition of `??`. Inside the window, the code walks for the first sample strictly after t ; an exact hit on a sample time returns that sample directly `model/ThrustCurve.cpp:61`, and running off the end returns the last sample’s thrust `model/ThrustCurve.cpp:67`. The interior case is linear interpolation, in the code’s own names:

$$\text{slope} = \frac{\text{thrustEnd} - \text{thrustStart}}{\text{tEnd} - \text{tStart}}, \quad F(t) = \text{thrustStart} + (t - \text{tStart}) \cdot \text{slope} \quad (1)$$

`model/ThrustCurve.cpp:85`. The subtlety is the interval’s *start* ([Listing 1](#)): normally the previous sample — but real catalog curves often begin at $t > 0$ rather than at the origin, and then there is no previous sample to read.

```
// Interval start is the previous sample -- unless t precedes the first sample (i == begin), where
// the curve has no (0,0) origin and we ramp from it. std::prev(begin()) here would read out of
// bounds.
double tStart = 0.0;
double thrustStart = 0.0;
if(i != thrustCurve.cbegin())
{
    tStart = std::prev(i)->first;
    thrustStart = std::prev(i)->second;
}
const double tEnd = i->first;
const double thrustEnd = i->second;
const double slope = (thrustEnd - thrustStart) / (tEnd - tStart);
return thrustStart + (t - tStart) * slope;
```

Listing 1. The implicit-origin ramp `model/ThrustCurve.cpp:73`. When the query time precedes the first measured sample, the curve interpolates from an implicit $(0, 0)$ origin instead of dereferencing `std::prev(begin())`.

Why the origin ramp exists

Curves whose first sample sits at $t > 0$ used to send the interpolation walk through `std::prev(begin())` — an out-of-bounds heap read whose value depended on allocator state. The regression test preserves the symptom in its comment: “an order-dependent, non-reproducible apogee” `model/tests/ThrustCurveTests.cpp:41`, and pins the fix’s exact ramp — first sample at $t = 0.05$, oracles $F(0) = 0$, $F(0.025) = 5$, $F(0.05) = 10$ `model/tests/ThrustCurveTests.cpp:46`. **OK**

One level up, `MotorModel::getThrust(simTime)` adds the burnout latch: once `simTime` exceeds `thrust.getMaxTime() + ignitionTime` the motor returns zero forever, logging “motor burnout occurred” exactly once through the `burnOutOccurred` flag `model/MotorModel.cpp:72`. Otherwise it delegates the *absolute* simulation time straight to the curve `model/MotorModel.cpp:82` — correct only because ignition always happens at $t = 0$ ([Section 1.4](#)).

1.3 The mass model: constant- I_{sp} depletion

Why mass flow is proportional to thrust

A rocket motor produces thrust by expelling mass: $F = \dot{m} v_e$, where v_e is the effective exhaust velocity. If v_e is constant over the burn — the standard hobby-simulator assumption, equivalent to constant specific impulse $I_{sp} = v_e/g_0$ — the propellant depletion rate is proportional to instantaneous thrust: $\dot{m}_{prop}(t) = -F(t)/(g_0 I_{sp})$. Substituting the definition $I_{sp} = I_{tot}/(g_0 m_{prop,0})$,

where $m_{prop,0}$ is the initial propellant mass, makes I_{sp} cancel entirely:

$$\dot{m}_{prop}(t) = -F(t) \frac{m_{prop,0}}{I_{tot}}. \quad (2)$$

The proportionality constant is exactly the one that makes the propellant integrate to $m_{prop,0}$ when the thrust integrates to I_{tot} : hard-burning moments consume propellant fast, the tail-off slowly, and the books balance at burnout by construction. What constant I_{sp} ignores — chamber-pressure variation, mass expelled by delay and ejection charges — this model ignores too.

QtRocket implements Eq. (2) as a precomputed table. `computeMassCurve()` `model/MotorModel.cpp:125` first splits the loaded motor into casing and propellant, `emptyMass = totalWeight - propWeight` `model/MotorModel.cpp:127`, computes `isp = totalImpulse/(g0 * propWeight)` with $g_0 = 9.80665 \text{ m/s}^2$ `model/MotorModel.cpp:130` `utils/math/Constants.h:10` — a member that, as Eq. (2) predicts, cancels out of the depletion arithmetic and is in fact never read anywhere (??) — then builds the 128-point table (Listing 2). With $\Delta t = \text{burnTime}/127$ `model/MotorModel.cpp:135` and `propMass` seeded to `propWeight` `model/MotorModel.cpp:137`, the loop is a left Riemann sum over Eq. (2):

$$\text{propMass}_{i+1} = \text{propMass}_i - F(t_i) \Delta t \frac{\text{propWeight}}{\text{totalImpulse}}, \quad t_i = i \Delta t \quad (3)$$

`model/MotorModel.cpp:141`. The 128th point is then forced to exact depletion: `massCurve.push_back(std::make_pair(data.burnTime, 0.0));` `model/MotorModel.cpp:143`. Pinning the endpoint zeroes out whatever residue the Riemann sum accumulated, so the tabulated propellant mass is first-order-approximate between samples but *exact* at ignition and burnout — post-burn mass is exactly `emptyMass`, bit for bit, which is what lets the part tree’s composite cache freeze stably after burnout (??).

```
// Precompute the mass curve as a lookup table: keeps repeated getMass()/getThrust() calls within
// one time step consistent, and speeds up the sim. 128 sample points (RASP files cap at 32).
massCurve.reserve(128);
double timeStep = data.burnTime / 127.0;
double t = 0.0;
double propMass{data.propWeight};
for(std::size_t i = 0; i < 127; ++i)
{
    massCurve.push_back(std::make_pair(t + i*timeStep, propMass));
    propMass -= thrust.getThrust(t + i*timeStep) * timeStep * data.propWeight / data.totalImpulse;
}
massCurve.push_back(std::make_pair(data.burnTime, 0.0));
```

Listing 2. The 128-point depletion table `model/MotorModel.cpp:132`: a left Riemann sum of the constant- I_{sp} law with the final point pinned to exact depletion. Note what is absent — the vector is reserved but never cleared.

`getMass(simTime)` serves three regimes — “empty mass + remaining propellant mass” `model/MotorModel.cpp:30`. Before ignition: `totalWeight`, the loaded motor `model/MotorModel.cpp:32`. During the burn (`simTime - ignitionTime ≤ burnTime` `model/MotorModel.cpp:36`): walk the table — an entry matching within 4 units in the last place (`utils::math::floatingPointEqual` `utils/math/`

`UtilityMathFunctions.h:21`) returns directly `model/MotorModel.cpp:43`, otherwise the bracketing interval interpolates exactly as the thrust curve does `model/MotorModel.cpp:52`. After: `emptyMass model/MotorModel.cpp:63`. The walk carries no end-iterator guard `model/MotorModel.cpp:41`; it is safe because the branch guard caps `thrustTime` at `burnTime` and the table's last entry is the literal pair `(burnTime, 0.0)`, so at the boundary the exact-hit test fires on the final element before the iterator can run off the end.

One asymmetry deserves plain statement: `getMass` gates the burn on the metadata's `burnTime` while `getThrust` gates on the curve's own `maxTime` `model/MotorModel.cpp:36` `model/MotorModel.cpp:72` — two clocks that agree only if the metadata is honest. RASP guarantees agreement by construction, deriving `burnTime` from the last sample time `model/RASPLoader.cpp:210`; `RockSim` reads `burn-time` as an independent file attribute `model/RSEDatabaseLoader.cpp:54`, so a malformed file could make thrust and mass disagree about when the burn ends.

Curve before metadata

`setMetaData()/moveMetaData()` trigger `computeMassCurve()` `model/MotorModel.cpp:85`, and the mass curve integrates the thrust curve — so the curve must be attached first, or the mass table silently integrates the default `(0, 0)` curve and the motor never loses mass. The rule is stated and obeyed at every construction site — both file loaders `model/RSEDatabaseLoader.cpp:96` `model/RASPLoader.cpp:242`, the REST client `model/ThrustCurveClient.cpp:328`, and the XML database loader: “Order matters: `setMetaData()` triggers `computeMassCurve()`, which integrates the thrust curve, so the curve must be in place first” `model/MotorModelDatabase.cpp:303`. It is enforced by convention and comment, not by the type system.

A latent trap: `massCurve` is never cleared

`computeMassCurve()` calls `massCurve.reserve(128)` and appends `model/MotorModel.cpp:134`; it never clears. Calling `setMetaData()` twice on the same instance would leave the stale 128 points in front of the new ones, and `getMass()`'s front-to-back walk would read the stale curve. No current call path does this — every loader builds a fresh instance, and `part::Motor::setMotorModel` replaces the whole owned object `model/parts/Motor.cpp:18` — but a future metadata-editing feature that mutates a live `MotorModel` in place would trip it.

1.4 Ignition

Ignition is a boolean latch, set exactly once per flight. The only production call site of `startMotor` is `RocketModel::launch(): motorPart->getMotorModel().startMotor(0.0); model/RocketModel.cpp:108`, invoked by `QtRocket::launchRocket()` immediately before the propagation loop `QtRocket.cpp:54`. `startMotor` sets `ignitionOccurred` and records `ignitionTime` `model/MotorModel.h:328`; the member's declaration warns that `ignitionTime` is read by the burnout test even before ignition, so garbage there would make “the flight depend on stack/heap layout” `model/MotorModel.h:341` — it is zero-initialized in the class definition. From ignition onward, thrust flows to `RocketModel::getForces` `model/RocketModel.cpp:71` and mass flows through the `part::Motor` leaf into the composite walk `model/RocketModel.cpp:36` (??; ?? covers the loop that queries both).

The ignition-time seam is half-wired

`startMotor(startTime)` accepts any ignition time, and both the mass lookup and the burnout latch honor it `model/MotorModel.cpp:36` `model/MotorModel.cpp:72`. But `MotorModel` never forwards

the ignition time to its curve: `ThrustCurve::setIgnitionTime` has zero call sites in the entire tree `model/ThrustCurve.h:31`, its body carries a commented-out `//maxTime += ignitionTime` `model/ThrustCurve.cpp:44`, and `getThrust` passes *absolute* simulation time to the curve `model/MotorModel.cpp:82`, whose own `ignitionTime` is permanently zero. Consequence: a hypothetical `startMotor(t0)` with $t_0 \neq 0$ — an air-start or staged ignition — would shift the mass curve and the burnout window by t_0 while thrust samples were still looked up at absolute time. Thrust and mass would disagree for the whole burn. The hazard is latent solely because the single production caller passes `0.0` `model/RocketModel.cpp:108`; a naive future consumer of the dormant plumbing inherits it intact. **FAIL**

Two neighboring fields share this dormant status, stated once here and inventoried in `??`. The `delays` vector is parsed by all three ingest paths and persisted through the XML round trip, but nothing in `sim/`, `cli/`, or `gui/` consumes it — there is no ejection-charge or recovery-deployment model, so the plugged sentinel `1000` `model/MotorModel.h:303` guards behavior that does not yet exist. And `isp` is computed on every metadata set `model/MotorModel.cpp:130` and read by nothing; the depletion formula embeds it implicitly (Eq. (2)).

1.5 MotorModelDatabase: one keyed surface

`model::MotorModelDatabase` `model/MotorModelDatabase.h:68` is the subsystem’s single client-facing surface: a non-copyable, non-movable `model/MotorModelDatabase.h:75` store owned as a `shared_ptr` by the QtRocket controller `QtRocket.cpp:51`. Its heart is disarmingly small — `std::map<std::string, MotorModel>` keyed by common name, the in-tree comment admitting “The ‘database’ is really just a map. :)” `model/MotorModelDatabase.h:127`.

Clients reach motors only through the database

The loaders are constructed stack-locally inside `importRSEFile/importRASPFFile` — “owned here, never seen by GUI/CLI” `model/MotorModelDatabase.cpp:63` — and the REST client hides behind the private `ThrustCurveAPI` injection constructor `model/MotorModelDatabase.h:114`. Every motor, whatever its source, funnels into the same keyed map with the same query model, and the front ends prove the rule: all GUI and CLI motor traffic goes through database methods (`gui/CannonballTab.cpp:137`, `gui/ThrustCurveMotorSelector.cpp:52`, `cli/Repl.cpp:315`), and a saved design references its motor by common name alone, re-resolved against whatever database the user has loaded `model/DesignSerializer.cpp:271` (`??`). One practical payoff: the loaders’ miss semantics — a default-constructed, zero-thrust, zero-mass `MotorModel` `model/RSEDatabaseLoader.cpp:41` — never reach application code, which sees the database’s `std::optional` instead `model/MotorModelDatabase.cpp:84`.

Keying and collisions. `addMotorModel` keys on `data.commonName` and assigns unconditionally: `motorModelMap[name] = m;` `model/MotorModelDatabase.cpp:50` — last write wins, replace-versus-add distinguished only in the logs `model/MotorModelDatabase.cpp:44`. The import methods report *net new* entries — the map’s size delta — so “same-name motors collapse onto one entry, and a re-import correctly reports 0” `model/MotorModelDatabase.cpp:68`. The bundled `data/Aerotech.rse` makes it concrete: 252 `<engine>` elements, but three codes (G77R, G79W, I65W) appear twice each, so a first import reports 249 and a second reports 0 — both pinned by test `tests/MotorDatabasePersistenceTests.cpp:348`.

Queries. `listMotors` takes a `MotorQuery` whose fields are all `std::optional` — unset does not constrain, so a default query matches everything `model/MotorModelDatabase.h:33`. The filters: exact match on manufacturer and impulse class `model/MotorModelDatabase.cpp:102`, case-sensitive substring on the common name `model/MotorModelDatabase.cpp:110`, and a diameter tolerance of ± 0.5 mm — motor diameters are nominal integer millimeters at least 1 mm apart, so the tolerance distinguishes sizes while forgiving doubles round-tripped through XML `model/MotorModelDatabase.cpp:106`; the boundary is pinned at 29.49 matching a 29 mm motor and 29.51 not `tests/MotorDatabasePersistenceTests.cpp:445`. Results are `MotorSummary` rows — a deliberately thin type that lets a UI populate a list without holding full, possibly not-yet-downloaded thrust curves; callers fetch the full model on selection `model/MotorModelDatabase.h:41`. Sorting comes free: `std::map` iterates in key order `model/MotorModelDatabase.cpp:116`.

Persistence: the .qmd format. `saveMotorDatabase` writes a Boost property-tree XML document rooted at `QtRocketMotorDatabase` with `version="0.1"` `model/MotorModelDatabase.cpp:184`: one `<motor>` element per map entry with twenty child fields — enums serialized through their `str()` wrappers `model/MotorModelDatabase.cpp:190`, delays as a CSV string whose writing loop is index-guarded because “the old `size()-1` form underflowed” on the empty delay vectors online-sourced motors carry `model/MotorModelDatabase.cpp:211`, and the thrust curve as `<thrust time=".." force=".."/>` children `model/MotorModelDatabase.cpp:222`. Numerics round-trip exactly because `property_tree` writes `max_digits10` `tests/MotorDatabasePersistenceTests.cpp:315`. `loadMotorDatabase` mirrors it: skip any node that is not `motor` `model/MotorModelDatabase.cpp:249`, default every missing field `model/MotorModelDatabase.cpp:255`, re-parse delays skipping empty tokens (so “4,,7,” yields {4, 7} `model/MotorModelDatabase.cpp:281`), rebuild curve-first `model/MotorModelDatabase.cpp:302`, and merge through `addMotorModel` `model/MotorModelDatabase.cpp:307` — a load is an import, with the same last-write-wins semantics. The full round trip is pinned end to end: import the real 249-motor Aerotech file, save, reload, compare field by field `tests/MotorDatabasePersistenceTests.cpp:297`. Listing 3 shows the format on a committed fixture.

```
<QtRocketMotorDatabase version="0.1">
  <MotorModels>
    <motor name="1/2A3">
      <availability>regular</availability>
      <avgThrust>3.0299999999999998</avgThrust>
      <burnTime>0.35999999999999999</burnTime>
      <certOrg>Uncertified</certOrg>
      <commonName>1/2A3</commonName>
      <designation>1/2A3</designation>
      <diameter>13</diameter>
      <impulseClass>A</impulseClass>
```

Listing 3. Opening of the committed .qmd fixture `tests/data/motors/estes_small.qmd:2`. Enums travel as their `str()` strings, numerics at `max_digits10`. Note the 1/2A3 motor carrying `<impulseClass>A</impulseClass>` — the server-verbatim class discussed in Section 1.7.

The typos are load-bearing

`CertOrg::str()` emits “Austrialian Model Rocket Society Inc.” `model/MotorModel.h:128` and “Unkown” `model/MotorModel.h:138` — misspelled, and deliberately preserved. The .qmd format’s losslessness rests on every enum wrapper’s `str()`→`toEnum()` round trip, so `toEnum` explic-

itly accepts the typo'd full names alongside the short codes: “The ‘Austrialian’/‘Unkown’ spellings match str()'s typo'd output” `model/MotorModel.h:143`. Correcting the spelling without a migration would orphan the AMRS and UNK values in every existing saved database; the round-trip test — every wrapper, every enum value — would catch the break `tests/MotorDatabasePersistenceTests.cpp:245`. Format-compatibility strings are part of the interface, typos included.

1.6 Three ingest paths

Three paths feed the map, each translating one interchange format to the `MetaData` contract of [Table 1](#). They differ sharply in error posture — [Table 2](#) contrasts them — but converge on the same assembly ritual: build the thrust curve, attach it, then move the metadata in.

	RockSim <code>.rse</code>	RASP <code>.eng</code>	thrustcurve.org REST
Parser	<code>RSEDatabaseLoader</code> , Boost <code>property_tree</code> XML <code>model/RSEDatabaseLoader.cpp:24</code>	<code>RASPLoader</code> , hand-rolled line parser <code>model/RASPLoader.cpp:253</code>	<code>ThrustCurveClient</code> , <code>jsoncpp</code> over <code>utils::CurlConnection</code> <code>model/ThrustCurveClient.h:99</code>
Error posture	lenient: nearly every attribute defaults <code>model/RSEDatabaseLoader.cpp:53</code> ; Type is the one non-defaulted read	strict: typed, line-numbered <code>std::runtime_error</code> on any violation <code>model/RASPLoader.cpp:65</code>	tolerant: malformed JSON → <code>nullopt</code> + log; server error field → empty-but-valid <code>model/ThrustCurveClient.cpp:37</code>
Weights in source	grams, ÷1000 <code>model/RSEDatabaseLoader.cpp:81</code>	kilograms, as-is <code>model/RASPLoader.cpp:237</code>	grams, ÷1000 <code>model/ThrustCurveClient.cpp:208</code>
Impulse class	derived from the motor code <code>model/RSEDatabaseLoader.cpp:73</code>	derived from the motor code <code>model/RASPLoader.cpp:233</code>	server field, verbatim <code>model/ThrustCurveClient.cpp:201</code>
burnTime	file attribute <code>model/RSEDatabaseLoader.cpp:54</code>	last sample time <code>model/RASPLoader.cpp:210</code>	server field <code>model/ThrustCurveClient.cpp:195</code>
Data discarded	per-sample mass/CG, comments <code>model/RSEDatabaseLoader.cpp:91</code>	—	all simfiles beyond the chosen one <code>model/ThrustCurveClient.cpp:67</code>

Table 2. The three ingest paths behind the database facade. All converge on kilogram weights, millimeter dimensions, and the curve-before-metadata assembly order; they diverge in how much they trust their input.

1.6.1 RockSim `.rse`: lenient XML

The `.rse` format is the XML engine-list format of the commercial RockSim simulator; QtRocket’s only bundled motor data, `data/Aerotech.rse`, uses it. The loader parses eagerly in its constructor via `boost::property_tree::read_xml` `model/RSEDatabaseLoader.cpp:24` and iterates every child of `engine-database.engine-list` without a key filter `model/RSEDatabaseLoader.cpp:27`; the format is assumed well-formed, and parse failures propagate as exceptions `model/MotorModelDatabase.h:86`. Metadata comes from XML attributes with defaults: `code` becomes the common name `model/RSEDatabaseLoader.cpp:56`, `initWt/propWt` divide by 1000 `model/RSEDatabaseLoader.cpp:81`, delays parse

from a CSV string defaulting to the plugged sentinel "1000" `model/RSEDatabaseLoader.cpp:59`. The one exception to leniency is accidental: `Type` is read with no default, so a file lacking that attribute throws from inside `property_tree` `model/RSEDatabaseLoader.cpp:85`.

What the loader *discards* matters as much as what it keeps. Each `<eng-data>` sample in an `.rse` file carries four attributes — time, force, and measured remaining mass and center-of-gravity (`m=`, `cg=`) — but only `t` and `f` are read `model/RSEDatabaseLoader.cpp:91`. QtRocket deliberately reconstructs motor mass from thrust via the constant- I_{sp} model of [Section 1.3](#) rather than consuming the file’s measured mass samples, and it models the motor’s own center of mass as fixed (the `part::Motor` leaf’s offset is `Vector3::Zero()` `model/parts/Motor.cpp:12`): propellant burn moves the *composite* center of mass only through motor mass loss, not through intra-motor CG travel. A modeling simplification, chosen with the data’s existence in plain view.

1.6.2 RASP .eng: strict text

RASP `.eng` is the line-oriented text format of the hobby’s oldest simulator lineage, and the loader treats it with the opposite posture: every violation is a typed, located error — lines are numbered from 1 as read `model/RASPLoader.cpp:260`, and every failure throws `std::runtime_error("RASP parse error on line N: message")` `model/RASPLoader.cpp:65`. The header must contain exactly seven whitespace-separated fields — code, diameter (mm), length (mm), delays, propellant weight (kg), total weight (kg), manufacturer `model/RASPLoader.cpp:145` — with positivity and `totalWeight ≥ propWeight` enforced `model/RASPLoader.cpp:157`. Delays are dash-separated (4-7); each token must be a non-negative integer or the case-insensitive `P/NONE`, which map to the plugged sentinel 1000 `model/RASPLoader.cpp:98`. Sample lines must hold exactly two finite numbers `model/RASPLoader.cpp:171`, times strictly increasing, thrust non-negative `model/RASPLoader.cpp:300`. A zero-thrust sample marks the end of a motor — a sample after it is the error “zero-thrust sample must be the final data point” `model/RASPLoader.cpp:289`, a motor that never reaches zero is “motor thrust data must end with a zero-thrust sample” `model/RASPLoader.cpp:314` — and a non-sample line after the final zero starts the next motor in a multi-motor file `model/RASPLoader.cpp:281`.

Because a RASP file carries no summary metadata beyond the header, the loader derives what the other formats read. After prepending the implicit origin if the first sample sits at $t > 0$ `model/RASPLoader.cpp:207` — the parse-time counterpart of [Listing 1](#)’s runtime ramp:

$$\text{burnTime} = t_{\text{last}}, \quad \text{totalImpulse} = \sum_{i \geq 1} \frac{1}{2} (F_{i-1} + F_i) (t_i - t_{i-1}), \quad \text{avgThrust} = \frac{\text{totalImpulse}}{\text{burnTime}} \quad (4)$$

— trapezoid integration of the samples `model/RASPLoader.cpp:189`, with `maxThrust` the sample maximum `model/RASPLoader.cpp:217` and both derived quantities checked positive `model/RASPLoader.cpp:212`. A spec-style Estes D12 test pins the pipeline end to end `model/tests/RASPLoaderTests.cpp:45`. No `.eng` file ships in `data/`; RASP coverage runs entirely on inline fixtures.

1.6.3 thrustcurve.org: REST behind a seam

The third path is the community motor-data service `thrustcurve.org`, reached over its REST (representational state transfer) API from base URL `https://www.thrustcurve.org/` `model/ThrustCurveClient.cpp:235` via the libcurl wrapper `utils::CurlConnection`, whose `get()` returns the body on success and an empty string on any transport failure `utils/CurlConnection.h`. Three endpoints are consumed. `GET api/v1/metadata.json` `model/ThrustCurveClient.cpp:268` yields the filterable vocabulary. `GET api/v1/search.json` `model/ThrustCurveClient.cpp:283` turns each criteria pair into a query parameter and, absent a caller-supplied limit, appends `maxResults=100`: the server default is 20,

and the comment explains the ceiling — each result costs one `download.json` round trip, performed synchronously on the caller’s (often GUI) thread `model/ThrustCurveClient.cpp:292`. The search parser maps each result into a `MetaData`: gram fields divided by 1000 `model/ThrustCurveClient.cpp:208`, the server’s `motorId` kept as `motorIdTC` `model/ThrustCurveClient.cpp:206`, the `impulseClass` field taken verbatim `model/ThrustCurveClient.cpp:201`, and `certOrg` and `delays` left at defaults under explicit `TODOs` `model/ThrustCurveClient.cpp:197` — the empty delay vectors those `TODOs` produce are what the `.qmd` CSV underflow guard exists for.

Finally, `GET api/v1/download.json?motorId=<id>&data=samples` `model/ThrustCurveClient.cpp:248` fetches the curve itself. A motor typically has several “simfiles” — alternative measured descriptions of the same burn, in arbitrary order — and the parser selects exactly one: the first RASP-format entry with samples wins immediately, else the first non-empty entry of any format is the fallback `model/ThrustCurveClient.cpp:84`. The in-code rationale is blunt: “their sample sets are alternative descriptions of the same burn, so we must pick exactly one... Concatenating them would interleave time series” `model/ThrustCurveClient.cpp:67`. A motor whose download produced no usable samples is still added — it keeps the default (0, 0) curve `model/ThrustCurveClient.cpp:324`. `searchMotors` assembles each full `MotorModel` curve-first `model/ThrustCurveClient.cpp:328` and warns when the server’s `matches` count exceeds the returned set — the truncation signal `model/ThrustCurveClient.cpp:312`.

The database wraps this behind two methods. `getOnlineSearchFacets()` maps the metadata to filter values, keeping only manufacturer *codes* because the search API keys on the code `model/MotorModelDatabase.cpp:148`. `searchOnline(q)` translates the source-agnostic `MotorQuery` into raw `SearchCriteria` `model/MotorModelDatabase.cpp:159` — diameter formatted with `std::format` so 29.0 travels as “29” `tests/MotorDatabasePersistenceTests.cpp:559`, and `nameContains` deliberately omitted, documented as ignored for online search `model/MotorModelDatabase.h:104` — then merges every returned motor into the map via `addMotorModel` so subsequent *local* queries see it `model/MotorModelDatabase.cpp:169`. An online search is an import.

The seam that makes this testable is `ThrustCurveAPI` `model/ThrustCurveClient.h:68`: a two-method pure-virtual interface that exists, per its own header, so the online wrappers can be tested with a fake source — “the injection point is private/friend-only in `MotorModelDatabase`, so application code never uses this directly” `model/ThrustCurveClient.h:63`. The public default constructor delegates with a null API `model/MotorModelDatabase.cpp:24`, the production `ThrustCurveClient` is constructed lazily on first online use `model/MotorModelDatabase.cpp:137`, and the befriended `MotorModelDatabaseTestAccess` lets the integration tests inject a `FakeThrustCurveAPI` returning canned motors `tests/MotorDatabasePersistenceTests.cpp:106`. The three JSON parsers are split out as free functions precisely so canned response bodies can exercise them with no HTTP at all `model/ThrustCurveClient.h:88` — ?? returns to both layers of this arrangement.

1.7 Impulse-class derivation and its one seam

The impulse-class string is search vocabulary — `listMotors` filters on it exactly `model/MotorModelDatabase.cpp:104` — and neither local file format carries it explicitly, so QtRocket derives it from the motor code. `MetaData::deriveImpulseClass` `model/MotorModel.cpp:97` skips leading whitespace, uppercases, checks the two fractional prefixes first — `starts_with("1/4A")` and `starts_with("1/2A")` `model/MotorModel.cpp:112` — and otherwise returns the first alphabetic character (“G80T” → “G”), or the empty string if none exists `model/MotorModel.cpp:117`. The `RockSim` and `RASP` paths share this one parser `model/RSEDatabaseLoader.cpp:73` `model/RASPLoader.cpp:233`, and tests pin the fractional cases through both `model/tests/RASPLoaderTests.cpp:102` `model/tests/RASPLoaderTests.cpp:174`.

Fractional classes are source-dependent

The `thrustcurve.org` path does not derive; it stores the server's `impulseClass` field verbatim `model/ThrustCurveClient.cpp:201` — and the committed fixture, captured from the real API, shows a 1/2A3 motor carrying `<impulseClass>A</impulseClass>` `tests/data/motors/estes_small.qmd:12` (Listing 3): the server reports the fractional-class motor under its parent letter. The same physical motor therefore lands in the database with class "1/2A" when imported from an `.rse` or `.eng` file and class "A" when fetched online, and a `listMotors` filter on `impulseClass = "1/2A"` `model/MotorModelDatabase.cpp:104` matches the former and misses the latter. The discrepancy is data-visible today in any mixed-source database. **FAIL**

That seam aside, the subsystem delivers what the rest of the document consumes: a keyed catalog with one query model, and per-motor $F(t)$ and $m(t)$ functions whose books balance exactly at ignition and at burnout. ?? showed how the mass function enters the composite tree through the `part::Motor` leaf; ?? follows both functions into the propagation loop, where thrust is sampled every integrator stage and mass gates the composite-property cache; ?? covers the design file that names a motor and expects this database to resolve it; and the dormant inventory — non-zero ignition, delays, `isp`, the unread cert-org and type facets — is consolidated with everything else in ??.